P2248R1
Enabling list-initialization for algorithms

Published Proposal, 2020-11-24

Issue Tracking:
Inline In Spec

Author:
Giuseppe D'Angelo

Audience:
SG18

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to default some template type parameters in certain Standard Library function templates, so that these functions become callable using list-initialization. These functions are algorithmic related functions, available in `<algorithm>`, `<numeric>` or in standard containers' headers.

Table of Contents

§ 1. Changelog

- R1
  - Clarified some possible sources of ABI incompatibility, which were (guiltly) not discussed in R0.
  - Added a reference and some discussion related to [P1997R1].
  - Added a reference to [algorithms.requirements].
  - Fixed misspellings / formatting.
- R0
  - First submission

§ 2. Tony Tables

Before

```
struct point { int x; int y; };

std::vector<point> v;

v.push_back({3, 4}); // No need to specify the "point" type here...

// ... but must spell out the type here:
// an initializer list in algorithms is not able to deduce the type
std::find(v.begin(), v.end(), point{3, 4});
std::lower_bound(v.begin(), v.end(), point{3, 4});
std::search_n(v.begin(), v.end(), 10, point{3, 4});
std::ranges::fill(v, point{3, 4});
erase(v, point{3, 4});
```

After

```
struct point { int x; int y; };

std::vector<point> v;

v.push_back({3, 4});

// With the proposed changes: can
// the type gets deduced from the
std::find(v.begin(), v.end(), {3,
std::lower_bound(v.begin(), v.end
std::search_n(v.begin(), v.end(),
std::ranges::fill(v, {3, 4});
erase(v, {3, 4});
```

§ 3. Motivation and Scope

List-initialization ([dcl.init.list]) has been introduced in C++11. When an initializer list is used as a function argument, overload resolution works in a specific way to find a suitable overload, possibly constructing a temporary object of the argument's type (see [over.ics.list]).

This language feature allows to write code like this:

```
void f(std::complex<double>);
f({1.23, 4.56}); // OK
```

In case of function templates, the deduction rules do not allow deduction in the general case, because a type cannot be determined ([temp.deduct.call]). It is however allowed to use an initalizer list in case the corresponding template type parameter has a default:

```
template <typename T> void g(T);
g({1, 2});     // ERROR: cannot deduce T


template <typename T = std::complex<double>> void h(T);
h({1.0, 2.0}); // OK, the default is used for deducing T
```

Default template type parameters have a somehow poor adoption in the Standard Library. They are used for instance in `std::exchange`:

```
namespace std {
  // [utility.exchange]
  template<class T, class U = T>
  constexpr T exchange(T& obj, U&& new_val);
}

// thanks to U defaulted to T, we can write this:
std::complex<double> old_val;
auto new_val = std::exchange(old_val, {3.0, 4.0}); // OK

// old_vec is moved onto new_vec, and reset to a default-constructed state
std::vector<int> old_vec;
auto new_vec = std::exchange(old_vec, {}); // OK
```

Unfortunately, they are not used in a number of other places, such as some standard algorithms, although reasonable defaults could still be determined based on other available template parameters.

Let's consider C++20's `std::erase` for `std::vector`:

```
namespace std {
  // [vector.erasure]
  template<class T, class Allocator, class U>
  constexpr typename vector<T, Allocator>::size_type
  erase(vector<T, Allocator>& c, const U& value);
}

std::vector<std::complex<double>> numbers;
numbers.push_back({1.0, 2.0}); // OK
erase(numbers, {1.0, 2.0});    // ERROR: cannot deduce U
```

Since `U` does not have a default, one cannot use list initialization for the `value` parameter (see the examples in the § 2 Tony Tables above). Instead, `U` could be reasonably defaulted to `T` in `erase`'s template argument list (just like `std::exchange`) and therefore enabling the above syntax. This is one of the changes we are proposing.

Similarly, an algorithm like `std::find` is also lacking a default:

```
namespace std {
  // [alg.find]
  template<class InputIterator, class T>
  constexpr InputIterator
  find(InputIterator first, InputIterator last, const T& value);
}

std::vector<std::complex<double>> numbers;
numbers.push_back({1.0, 2.0});                          // OK
std::find(vector.begin(), vector.end(), {1.0, 2.0}); // ERROR
```

In this case `T` could be reasonably defaulted to be the value type of the input iterator, that is, `typename std::iterator_traits<InputIterator>::value_type`. We are also proposing this change.

> In a nutshell: we are proposing to add a default template type parameter to the "algorithmic" function templates in the Standard Library:
>
> 1. which have a "value"-like function argument; and
> 2. for which a reasonable default type can be inferred from the other template type parameters.
>
> This enables users to call these functions by using list-initialization for the "value"-like arguments.

4. Impact On The Standard

The proposed changes only affect existing Standard Library function templates. The impact is mostly positive: syntax that was ill-formed before becomes well-formed. There is a chance of incompatibility w.r.t. user code, but [SD-8] allows the kind of changes we are proposing.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

The first part of [P2218R0] is related to this proposal. [P2218R0] proposes to default the template type parameter of `optional::value_or`. We are not proposing the same, as we are limiting the scope of this paper to algorithms; we can't help but notice however that the underlying rationale is exactly the same (enabling list-initialization for a "value" parameter).

[P1997R1] proposes some changes to the core language in order to make array types copiable, assignable, etc. We are not proposing any changes to the language; however it's interesting to note that, if [P1997R1] is adopted, then using arrays as value types in containers and algorithms may become more widespread. (Right now, array types are usable in containers/algorithms in extremely limited ways.) The present proposal would then make it possible to use list-initialization in order to create arrays as the arguments of algorithmic-like functions, for instance like this:

```
// in principle, possible under P1997R1
std::vector<int[2]> v(42);
v.push_back({1, 2});

// in principle, possible under P1997R1 + this proposal
// (T defaults to int[2], second parameter is deduced as const int(&)[2],
// then aggregate initialization is selected)
std::ranges::fill(v, {123, 456});
```

5. Design Decisions

5.1. Is this API compatible?

We are fairly certain that the proposed changes keep API compatibility.

Technically speaking, defaulting a template type parameter of a function template is a detectable change. Here's an example (many thanks to of Arthur O'Dwyer, who provided a much simpler one than ours):

```
1 template <typename A, typename B /* = A */> int f(A, B) { return 1; }
2
3 int f(int, int *) { return 2; }
4
5 int x = f(0, {0});
```

In this code `x` is equal to 2 (there is only one candidate). If we change line 1 so that `B` is defaulted to `A` (removing the comment marks), then that function template is able to provide a better candidate and `x` will be 1 instead.

We do not think this is actually a meaningful problem. For this to affect client code, such code must be performing a function call that somehow is including a Standard Library function during its overload resolution, and then selecting some other function as the best candidate. If that's the case, any change in the Standard Library (such as adding an overload) could affect the overload resolution and select a different candidate.

We believe that [SD-8] gives us the "safety blanket" of doing the changes we are proposing: we are explicitly allowed to default template parameters.

§ 5.2. Is this ABI compatible?

This proposal deals with two kind of changes to function templates.

§ 5.2.1. Changes to consistent container erasure and non-range-based algorithms

Here we are simply proposing to default a template type parameter in the template parameter list of the related functions. This is entirely ABI compatibile (we're dealing with *function* templates, and not class templates).

§ 5.2.2. Changes to range-based algorithms

These changes require reordering the template argument list (see the discussion in § 5.3 Range-based algorithms). This is unfortunately not 100% ABI compatibile, as the instantiations of these functions would change their mangling.

With the reorderings proposed, we feel it's very unlikely that this incompatibility would cause a problem in practice. A user would have to call range-based algorithm using the same type for the template parameters that got reordered. For instance, for `std::ranges::remove`, one would need to use the same type as the projection type and for the value to remove (see § 6.3.6 [alg.remove]) in order to get a clash.

In the absence of such a type clash, then the problem would be averted: old code would keep using the old mangled name, and code compiled against the new version would use the new name. (The final binary may end up with multiple copies of the instantiated function, under different symbols.)

There would still be the chance of an ABI break for user code using explicit template instantiations of one of these functions. The TU defining the instantiation would then be incompatible with any other TU that uses a different version of these functions (and merely declares the instantation).

We are not sure that user code is actually allowed to use explicit instantiations of Standard Library functions. Such a usage seems to violate the [SD-8] directives, specifically the right of the Standard Library to:

> - Make changes to existing interfaces in a fashion that will be backward compatible, if those interfaces are solely used to instantiate types and invoke functions. Implementation details (the primary name of a type, the implementation details for a function callable) may not be depended upon.
>
>   For example, we may change implementation details for standard function templates so that those become callable function objects. If user code only invokes that callable, the behavior is unchanged.

ISSUE 1 What does LEWG(I) think about this?

§ 5.3. Range-based algorithms

The current specification in the Standard for Range-based algorithms does not allow for a "straightforward" defaulting, like the non-range-based variants.

`ranges::find` has this synopsis in [algorithm.syn]:

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity>
  requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
  constexpr I find(I first, S last, const T& value, Proj proj = {});
```

```
template<input_range R, class T, class Proj = identity>
  requires indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T*>
  constexpr borrowed_iterator_t<R>
    find(R&& r, const T& value, Proj proj = {});
```

Both declarations do not allow for `T` to be defaulted given the ordering of template type parameters. `T` should be defaulted to something like `typename projected<I, Proj>::value_type` in the first version, and to `typename projected<iterator_t<R>, Proj>::value_type` in the second version. This is not possible: the template parameter `Proj` appears *after* `T` in the template parameter list.

In order to fix this, we need to reorder the template parameters.

Some other algorithms need a slightly more elaborate change. Let's consider `ranges::fill`:

```
template<class T, output_iterator<const T&> O, sentinel_for<O> S>
  constexpr O fill(O first, S last, const T& value);

template<class T, output_range<const T&> R>
  constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
```

Here the output iterator `O` is defined in terms of `T`, making it impossible to default `T` in terms of `O` instead. The proposed solution in this case is reorder the template parameter list and move the constraint out of it, in a `requires` clause. This will keep the pre-existing semantics while allowing `T` to be defaulted.

To summarize, the changes that we are proposing to the range-based algorithms go in this direction: they move template parameters and/or constraints in order to be able to give a meaningful default type to the "value"-like function parameters.

We believe that [SD-8] allows us to do so under the interpretation that the exact spelling or ordering of template type arguments and constraints is part of the "implementation details" that users cannot depend upon. There will be no changes for users who simply *call* these functions, letting the template arguments to be deduced.

Furthermore, specifically for algorithms, users are already not allowed to rely on the number and the order of deducible template parameters. [algorithms.requirements] / 15 says:

> The number and order of deducible template parameters for algorithm declarations are unspecified, except where explicitly stated otherwise.
> [Note 3: Consequently, an implementation can reject calls that specify an explicit template argument list. — end note]

Please note, however, that in spite of the of SD-8 rules, this kind of changes may be ABI incompatible. See § 5.2 Is this ABI compatible? for a discussion.

§ 6. Technical Specifications

All the proposed changes are relative to [N4868].

§ 6.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_default_template_type_for_algorithm_values YYYYMML  // also in
    // <algorithm>, <ranges>, <numeric>,
    // <string>, <deque>, <list>, <forward_list>, <vector>
```

with the value specified as usual (year and month of adoption).

§ 6.2. Consistent Container Erasure

Modify [string.syn] and [string.erasure]:

```
template<class charT, class traits, class Allocator, class U = charT>
  constexpr typename basic_string<charT, traits, Allocator>::size_type
    erase(basic_string<charT, traits, Allocator>& c, const U& value);
```

Modify [deque.syn] and [deque.erasure]:

```
template<class T, class Allocator, class U = T>
  typename deque<T, Allocator>::size_type
    erase(deque<T, Allocator>& c, const U& value);
```

Modify [forwardlist.syn] and [forward.list.erasure]:

```
template<class T, class Allocator, class U = T>
  typename forward_list<T, Allocator>::size_type
    erase(forward_list<T, Allocator>& c, const U& value);
```

Modify [list.syn] and [list.erasure]:

```
template<class T, class Allocator, class U = T>
  typename list<T, Allocator>::size_type
    erase(list<T, Allocator>& c, const U& value);
```

Modify [vector.syn] and [vector.erasure]:

```
template<class T, class Allocator, class U = T>
  constexpr typename vector<T, Allocator>::size_type
    erase(vector<T, Allocator>& c, const U& value);
```

§ 6.3. Algorithms

Modify both the `<algorithm>` synopsis ([algorithm.syn]) as well as the algorithm signatures in each and every referenced section:

§ 6.3.1. [alg.find]

```
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type>
  constexpr InputIterator find(InputIterator first, InputIterator last,
                               const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<I
  ForwardIterator find(ExecutionPolicy&& exec,
                       ForwardIterator first, ForwardIterator last,
                       const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = t
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr I find(I first, S last, const T& value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = typename projected<iter
    requires indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
```

```
    constexpr borrowed_iterator_t<R>
      find(R&& r, const T& value, Proj proj = {});
}
```

§ 6.3.2. [alg.count]

```
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr typename iterator_traits<InputIterator>::difference_type
    count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<I
  typename iterator_traits<ForwardIterator>::difference_type
    count(ExecutionPolicy&& exec,
          ForwardIterator first, ForwardIterator last, const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = t
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr iter_difference_t<I>
      count(I first, S last, const T& value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = typename projected<itera
    requires indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr range_difference_t<R>
      count(R&& r, const T& value, Proj proj = {});
}
```

§ 6.3.3. [alg.search]

```
template<class ForwardIterator, class Size, class T = typename iterator_traits<ForwardItera
  constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
             Size count, const T& value);
template<class ForwardIterator, class Size, class T = typename iterator_traits<ForwardItera
  constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
             Size count, const T& value, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T = typename itera
  ForwardIterator
    search_n(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             Size count, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T = typename itera
         class BinaryPredicate>
  ForwardIterator
    search_n(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             Size count, const T& value,
             BinaryPredicate pred);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T,
           class Pred = ranges::equal_to, class Proj = identity, class T = typename project
    requires indirectly_comparable<I, const T*, Pred, Proj>
    constexpr subrange<I>
      search_n(I first, S last, iter_difference_t<I> count,
               const T& value, Pred pred = {}, Proj proj = {});
```

```
  template<forward_range R, class T, class Pred = ranges::equal_to,
           class Proj = identity, class T = typename projected<iterator_t<R>, Proj>::value_
    requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
    constexpr borrowed_subrange_t<R>
      search_n(R&& r, range_difference_t<R> count,
               const T& value, Pred pred = {}, Proj proj = {});
}
```

§ 6.3.4. [alg.replace]

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr void replace(ForwardIterator first, ForwardIterator last,
                         const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  void replace(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last,
               const T& old_value, const T& new_value);
template<class ForwardIterator, class Predicate, class T = typename iterator_traits<Forward
  constexpr void replace_if(ForwardIterator first, ForwardIterator last,
                            Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class Predicate, class T = typename
  void replace_if(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last,
                  Predicate pred, const T& new_value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T1, class T2, class Proj = identity, _
    requires indirectly_writable<I, const T2&> &&
             indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
    constexpr I
      replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
  template<input_range R, class T1, class T2, class Proj = identity, class T1 = typename pr
    requires indirectly_writable<iterator_t<R>, const T2&> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T1*>
    constexpr borrowed_iterator_t<R>
      replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = t
           indirect_unary_predicate<projected<I, Proj>> Pred>
    requires indirectly_writable<I, const T&>
    constexpr I replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = typename projected<iter
           indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires indirectly_writable<iterator_t<R>, const T&>
    constexpr borrowed_iterator_t<R>
      replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});
}
```

```
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Outp
  constexpr OutputIterator replace_copy(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T = t
  ForwardIterator2 replace_copy(ExecutionPolicy&& exec,
                                ForwardIterator1 first, ForwardIterator1 last,
                                ForwardIterator2 result,
                                const T& old_value, const T& new_value);
template<class InputIterator, class OutputIterator, class Predicate, class T = typename ite
  constexpr OutputIterator replace_copy_if(InputIterator first, InputIterator last,
```

```
                                        OutputIterator result,
                                        Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class Predicate, class T = typename iterator_traits<ForwardIterator2>::value_type>
  ForwardIterator2 replace_copy_if(ExecutionPolicy&& exec,
                                   ForwardIterator1 first, ForwardIterator1 last,
                                   ForwardIterator2 result,
                                   Predicate pred, const T& new_value);
```

```
namespace ranges {
template<class I, class O>
  using replace_copy_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class T1, class T2,
         output_iterator<const T2&> O, class Proj = identity>
template<input_iterator I, sentinel_for<I> S, class O,
         class Proj = identity, class T1 = typename projected<I, Proj>::value_type, class T
  requires indirectly_copyable<I, O> &&
           indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
           output_iterator<O, const T2&>
  constexpr replace_copy_result<I, O>
    replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
                 Proj proj = {});
template<input_range R, class T1, class T2, output_iterator<const T2&> O,
         class Proj = identity>
template<input_range R, class O, class Proj = identity,
         class T1 = typename projected<iterator_t<R>, Proj>::value_type, class T2 = iter_va
  requires indirectly_copyable<iterator_t<R>, O> &&
           indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T1*> &&
           output_iterator<O, const T2&>
  constexpr replace_copy_result<borrowed_iterator_t<R>, O>
    replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                 Proj proj = {});

template<class I, class O>
  using replace_copy_if_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class T, output_iterator<const T&> O,class O,
         class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
  requires indirectly_copyable<I, O> && output_iterator<O, const T&>
  constexpr replace_copy_if_result<I, O>
    replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                    Proj proj = {});
template<input_range R, class T, output_iterator<const T&> O,class O, class T = range_value
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
  requires indirectly_copyable<iterator_t<R>, O> && output_iterator<O, const T&>
  constexpr replace_copy_if_result<borrowed_iterator_t<R>, O>
    replace_copy_if(R&& r, O result, Pred pred, const T& new_value,
                    Proj proj = {});
}
```

§ **6.3.5. [alg.fill]**

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  void fill(ExecutionPolicy&& exec,
            ForwardIterator first, ForwardIterator last, const T& value);
template<class OutputIterator, class Size, class T = typename iterator_traits<ForwardIterat
  constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);
```

```
template<class ExecutionPolicy, class ForwardIterator,
         class Size, class T = typename iterator_traits<ForwardIterator>::value_type>
  ForwardIterator fill_n(ExecutionPolicy&& exec,
                         ForwardIterator first, Size n, const T& value);
```

```
namespace ranges {
  template<class T, output_iterator<const T&> O, sentinel_for<O> S>
  template<class O, sentinel_for<O> S, class T = iter_value_t<O>>
    requires output_iterator<O, const T&>
    constexpr O fill(O first, S last, const T& value);
  template<class T, output_range<const T&> R>
  template<class R, class T = range_value_t<R>>
    requires output_range<R, const T&>
    constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
  template<class T, output_iterator<const T&> O>
  template<class O, class T = iter_value_t<O>>
    requires output_iterator<O, const T&>
    constexpr O fill_n(O first, iter_difference_t<O> n, const T& value);
}
```

§ **6.3.6. [alg.remove]**

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                                   const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  ForwardIterator remove(ExecutionPolicy&& exec,
                         ForwardIterator first, ForwardIterator last,
                         const T& value);
```

```
namespace ranges {
  template<permutable I, sentinel_for<I> S, class T, class Proj = identity, class T = typen
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr subrange<I> remove(I first, S last, const T& value, Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = typename projected<it
    requires permutable<iterator_t<R>> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_subrange_t<R>
      remove(R&& r, const T& value, Proj proj = {});
}
```

```
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Inpu
  constexpr OutputIterator
    remove_copy(InputIterator first, InputIterator last,
                OutputIterator result, const T& value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class T = typename iterator_traits<ForwardIterator1>::value_type>
  ForwardIterator2
    remove_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class T,
           class Proj = identity, class T = typename projected<I, Proj>::value_type>
    requires indirectly_copyable<I, O> &&
```

```
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr remove_copy_result<I, O>
      remove_copy(I first, S last, O result, const T& value, Proj proj = {});
  template<input_range R, weakly_incrementable O, class T, class Proj = identity, class T =
    requires indirectly_copyable<iterator_t<R>, O> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr remove_copy_result<borrowed_iterator_t<R>, O>
      remove_copy(R&& r, O result, const T& value, Proj proj = {});
}
```

§ 6.3.7. [alg.binary.search]

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr I lower_bound(I first, S last, const T& value, Comp comp = {},
                            Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = typename projected<it
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr borrowed_iterator_t<R>
      lower_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr I upper_bound(I first, S last, const T& value, Comp comp = {}, Proj proj = {}
  template<forward_range R, class T, class Proj = identity, class T = typename projected<it
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
```

```
    constexpr borrowed_iterator_t<R>
      upper_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr subrange<I>
      equal_range(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = typename projected<it
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr borrowed_subrange_t<R>
      equal_range(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr bool binary_search(I first, S last, const T& value, Comp comp = {},
                                 Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = typename projected<it
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr bool binary_search(R&& r, const T& value, Comp comp = {},
                                 Proj proj = {});
}
```

§ 6.4. Numeric algorithms

Modify both the `<numeric>` synopsis ([numeric.ops.overview]) as well as the numeric algorithm signatures in each and every referenced section:

6.4.1. [accumulate]

```
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr T accumulate(InputIterator first, InputIterator last, T init);
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr T accumulate(InputIterator first, InputIterator last, T init,
                         BinaryOperation binary_op);
```

6.4.2. [reduce]

```
template<class InputIterator>
  constexpr typename iterator_traits<InputIterator>::value_type
    reduce(InputIterator first, InputIterator last);
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr T reduce(InputIterator first, InputIterator last, T init);
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr T reduce(InputIterator first, InputIterator last, T init,
                     BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator>
  typename iterator_traits<ForwardIterator>::value_type
    reduce(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  T reduce(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator last, T init);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  T reduce(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator last, T init, BinaryOperation binary_op);
```

6.4.3. [exclusive.scan]

```
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Inpu
  constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result, T init);
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Inpu
  constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result, T init, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T = t
  ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T = t
         class BinaryOperation>
  ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, T init, BinaryOperation binary_op);
```

6.4.4. [inclusive.scan]

```
template<class InputIterator, class OutputIterator>
  constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
  constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result, BinaryOperation binary_op);
template<class InputIterator, class OutputIterator, class BinaryOperation, class T = typena
  constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                   OutputIterator result, BinaryOperation binary_op, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
  ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryOperation>
  ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryOperation, class T = typename iterator_traits<ForwardIterator1>::value
  ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, BinaryOperation binary_op, T init);
```

6.4.5. [transform.exclusive.scan]

```
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Inpu
         class BinaryOperation, class UnaryOperation>
  constexpr OutputIterator
    transform_exclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result, T init,
                             BinaryOperation binary_op, UnaryOperation unary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T = t
         class BinaryOperation, class UnaryOperation>
  ForwardIterator2
    transform_exclusive_scan(ExecutionPolicy&& exec,
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result, T init,
                             BinaryOperation binary_op, UnaryOperation unary_op);
```

6.4.6. [transform.inclusive.scan]

```
template<class InputIterator, class OutputIterator,
         class BinaryOperation, class UnaryOperation>
  constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op, UnaryOperation unary_op);
template<class InputIterator, class OutputIterator,
         class BinaryOperation, class UnaryOperation, class T = typename iterator_traits<In
```

```
  constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op, UnaryOperation unary_op, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryOperation, class UnaryOperation>
  ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result, BinaryOperation binary_op,
                             UnaryOperation unary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryOperation, class UnaryOperation, class T = typename iterator_traits<Fo
  ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result,
                             BinaryOperation binary_op, UnaryOperation unary_op, T init);
```

§ 6.4.7. [numeric.iota]

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value
  constexpr void iota(ForwardIterator first, ForwardIterator last, T value);
```

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

Special thanks to Stephan T. Lavavej for confirming that the omission of a default argument from the `erase()` functions was not a deliberate design choice of the consistent container erasure proposal. Thanks to Arthur O'Dwyer for the insightful discussions. Thanks to Will Wray for the discussions regarding supporting array types and the mentions of related paper work. Thanks to Michał Dominiak for pointing out the possible ABI issues but still encouraging to bring the paper forward.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4868]
Richard Smith. Working Draft, Standard for Programming Language C++. URL: http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2020/n4868.pdf

[P1997R1]
Krystian Stasiowski; Theodoric Stier. Relaxing Restrictions on Arrays. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1997r1.pdf

[P2218R0]
Marc Mutz. More flexible optional::value_or(). URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2218r0.pdf

[SD-8]
Titus Winter. SD-8: Standard Library Compatibility. URL: https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility

§ Issues Index

ISSUE 1 What does LEWG(I) think about this? ↵